

L02 The camera matrix, and why a pose needs a group

Everything in this course is a pose, a measurement, or an uncertainty about one of them.

The camera matrix, and why a pose needs a group

L02 · Unit 0

Everything in this course is a pose, a measurement, or an uncertainty about one of them.

Monday ran to *a point is a ray*. Today finishes the camera: we assemble $P = K[R \mid t]$ and calibrate a real lens. Then we ask what kind of object $[R \mid t]$ is, and get far enough to say what we need from it.

Two stages, and that is the whole camera

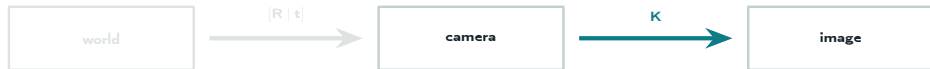


- **Extrinsics:** a rigid motion. Where the camera is, and which way it points. Six numbers, and they change every frame.
- **Intrinsics:** the projection, then metres to pixels. What the camera *is*. Five numbers, fixed until someone changes the lens.

Build each stage on its own, then multiply. That product is P .

The projection: camera frame to image

the pinhole ratio, in coordinates



Monday's similar triangles in coordinates, then rewritten up to scale so the division becomes a matrix:

$$\begin{pmatrix} x_i \\ y_i \\ 1 \end{pmatrix} = \frac{f}{z_c} \begin{pmatrix} x_c \\ y_c \\ 1 \end{pmatrix} \quad \Rightarrow \quad \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

The depth you divided by is the scale factor you threw away.

Projections

Careful

“Projection” names a linear map between vector spaces. The matrix above is the projection from the 3D camera frame to the 2D image. It is a different object from the projection matrix P .

Intrinsics: from metres to pixels

(x_i, y_i) is in metres on the sensor, measured from the optical axis. A pixel index is in pixels, counted from a corner. Three facts close the gap.

1. A pixel has a physical size (p_x, p_y) , so $f_x = F/p_x$ and $f_y = F/p_y$ convert the focal length F from millimetres into pixels.
2. The two origins differ. Shift by the **principal point** (c_x, c_y) , the pixel where the optical axis pierces the sensor.
3. If the sensor axes are not perpendicular there is a skew $s = f_x \tan \alpha$. It is 0 on every sensor you will meet.

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y$$

The intrinsic matrix

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Five degrees of freedom, all in pixels.

f_x	focal length in pixels, F/p_x
f_y	focal length in pixels, F/p_y
c_x	principal point, horizontal
c_y	principal point, vertical
s	skew, $f_x \tan \alpha$; zero in practice

K is upper triangular and invertible, so $K^{-1}\tilde{u}$ takes a pixel back to a ray direction. That inverse is how every depth sensor in L04 gets used.

A camera, with numbers

■ Build K for a real sensor, then project one point

Everything else today is this arithmetic with the unknowns moved around.

The principal point

and why it is not the image centre

Look straight out along the lens axis. The pixel you land on is (c_x, c_y) : the origin every projection is measured from, and the centre radial distortion grows away from. Lens and sensor are aligned to a tolerance, so it lands off the middle of the array.

■ **Sensor rectangle: pixel origin, optical axis, image centre**

Assuming the centre is a bias

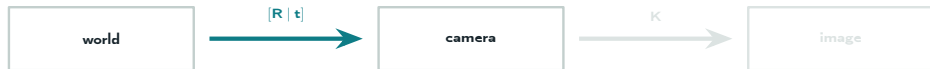
Trap

Setting $c_x = W/2$ is a **bias**, not an approximation. In the camera two slides ago the measured c_x was 642.1 and the image centre is 640: two pixels, applied to every observation, in the same direction, forever.

A bias does not average out with more data. It bends the map, and the residual stays clean while it does.

OpenCV returns c_x and c_y from calibration for this reason. Use the values it gives you rather than the image dimensions.

The extrinsics: world frame to camera frame



$$X_C = R X_W + t$$

A rotation and a translation, in ordinary Euclidean terms.

$$\begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

The bottom row is the reason for the fourth coordinate: it turns *add a vector* into *multiply by a matrix*, so the whole chain composes.

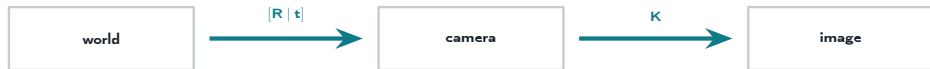
A pose, with numbers

World frame x forward, y left, z up. The camera sits at $C = (1, 0, 0.5)$ looking down world $+x$, optical frame z forward, x right, y down. A point five metres down the corridor:
 $X_w = (5, -0.5, 0.7)$.

■ Rows of R are the camera axes in world coordinates. Then move the point.

t is **not** where the camera is. The camera sits at the origin of its own frame, so C must map to zero: $0 = RC + t$. Equivalently, t is where the *world* origin lands in camera coordinates.

Compose the two stages



K turns the flattened point into pixels, so the chain is one product.

$$\tilde{x} \sim \underset{3 \times 3}{K} \underset{3 \times 4}{\begin{bmatrix} I & 0 \end{bmatrix}} \underset{4 \times 4}{\begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix}} \tilde{X}_w \implies \boxed{P = K [R | t]}$$

$w \tilde{x} = P \tilde{X}_w$: five intrinsic numbers, six extrinsic ones, one overall scale that does not matter.

The camera matrix

- Eleven degrees of freedom: twelve entries, minus the overall scale.
- The **null space** of P is the camera centre in world coordinates: $P\tilde{C} = 0$.
- It maps every world point to a pixel, and it is **not invertible**. A pixel is a ray, not a point.

Estimating P from correspondences

Given known 3D points X_i and the pixels they landed on, solve for P . Writing $p_1^\top, p_2^\top, p_3^\top$ for its rows, clearing the denominator gives two **linear** equations per point:

$$p_1^\top \tilde{X}_i - u_i p_3^\top \tilde{X}_i = 0, \quad p_2^\top \tilde{X}_i - v_i p_3^\top \tilde{X}_i = 0$$

■ **Unknowns, equations per measurement, minimum measurements**

Reprojection error

Six correspondences pin P down exactly. A seventh will not fit, because the data is noisy and the correspondences came from an algorithm with its own errors. What you want is the P that misses by least.

$$e = \frac{1}{N} \sum_{i=1}^N \|\hat{u}_i - u_i\|_2, \quad \hat{u}_i = \pi(P\tilde{X}_i)$$

- e is the mean miss over the N correspondences, and π is the perspective divide, $(a, b, c) \mapsto (a/c, b/c)$.
- $\hat{u}_i$ is where your estimate says point i should have landed; u_i is where it did.
- The units are **pixels**, which is why this is the currency of the whole field.
- A high residual often means a wrong correspondence rather than a bad P .

Where the linear model stops

real lenses bend rays

Correct distortion in **normalised** coordinates $(x, y) = (X/Z, Y/Z)$, *before* applying K. Write $r^2 = x^2 + y^2$.

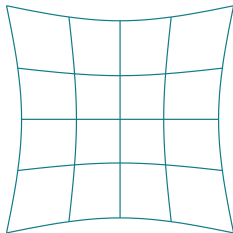
$$x_d = x \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \right), \quad y_d = y \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \right)$$

$$x_d = x + \left[2p_1 xy + p_2 \left(r^2 + 2x^2 \right) \right], \quad y_d = y + \left[p_1 \left(r^2 + 2y^2 \right) + 2p_2 xy \right]$$

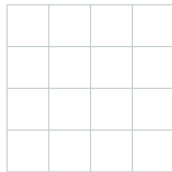
(x, y) where the ideal lens would put the point, (x_d, y_d) where the real one does, r the distance between the point and the optical axis. Radial on the top row, with k_1, k_2, k_3 ; tangential on the bottom, with p_1, p_2 .

Two radial coefficients are usually enough; k_3 is for wide angle. The model is no longer a matrix.

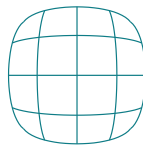
What radial distortion looks like



pincushion, $k_1 > 0$



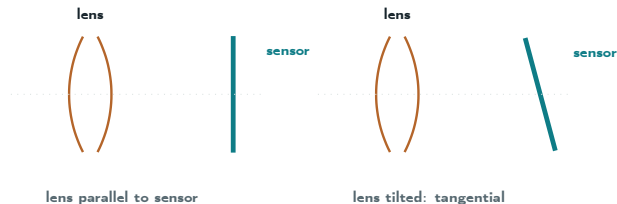
no distortion



barrel, $k_1 < 0$

$k_1 > 0$ pushes points outward, $k_1 < 0$ pulls them in. Straight lines photograph curved, and the effect grows with r , which is exactly where your calibration target's corners are.

Tangential distortion is a mounting error



It appears when the lens and the sensor are not parallel, which makes it a manufacturing tolerance rather than an optical property.

Trap

Distortion is applied *before* K. Getting the order backwards fits well at the centre and fails at the edges, which is where all your target corners are.

Calibration is an estimation problem

Photograph a planar checkerboard at several orientations. Every photo hands you the same thing you already know how to use: 3D points whose positions you know, and the pixels they landed on.

- **Unknown once:** K and the distortion coefficients k_1, k_2, k_3, p_1, p_2 .
- **Unknown per photo:** the pose $[R \mid t]$ of the board.
- **Known:** the board geometry, and every detected corner.

Choose all of them together to minimise the reprojection error, summed over every corner in every photo. Nothing new has been introduced except bookkeeping.

A calibration reported without its residual is a number somebody made up.

Half time

The camera is finished. $P = K[R \mid t]$, estimated from correspondences, refined against reprojection error, with a distortion model bolted on the front.

So far $[R \mid t]$ has been a block of twelve numbers we multiplied by. The rest of today asks what kind of object it is. Answering it takes all of next week.

Reference for everything above: the MathWorks calibration page, linked under **Resources**. Its notation is the notation on these slides.

Why not just use three numbers?

An orientation has three degrees of freedom, so store it as three numbers, roll, pitch and yaw, and then use the tools you already own. Add two of them. Average a batch. Differentiate. Run least squares.

Every one of those goes wrong. At pitch $\pm 90^\circ$ roll and yaw collapse onto the same axis and a degree of freedom disappears. Rotations do not commute, so three numbers cannot record which order you meant, and there are twelve Euler conventions with no default. The componentwise average of two orientations is not an orientation.

Reaching for a different representation does not rescue it. Quaternions trade the singularity for a sign ambiguity, and rotation matrices trade it for six constraints you now have to maintain by hand.

The labels were never the problem. Rotations are not a set you can do arithmetic on.

So we need two things

- A statement of which operations are **legal** here, and a guarantee they stay inside the set. That is what calling $SO(3)$ a **group** buys.
- A way to take derivatives, since every optimiser and every filter you own is built on *add a small correction*. That is what the **tangent space** buys.

Neither is abstraction for its own sake. Each one answers a question the last slide raised, and the rest of the hour is those two answers and their consequences.

What a group is

A **group** is a set of objects together with one way of combining two of them, obeying four rules.

- **Closure.** Combine two members and you get a member, never anything else.
- **Associativity.** $(ab)c = a(bc)$, so a chain needs no brackets.
- **Identity.** One member e that changes nothing: $ae = ea = a$.
- **Inverse.** Every member a has a partner a^{-1} with $aa^{-1} = e$.

Set and operation	Group?	Why
Integers, $+$	Yes	identity 0, inverse $-a$
Integers, $\times$	No	2 has no integer inverse
Shuffles of a deck	Yes	undo any shuffle, do nothing = identity
Rotations, composition	Yes	the one we need

What you should be able to do

- Write down K for a sensor given its focal length and pixel pitch, and project a point through P .
- Say what the five intrinsic and six extrinsic numbers are, and which of them calibration has to estimate.
- Say why three numbers cannot parameterise a rotation without a singularity somewhere.
- State the four group axioms and check them against a set you are handed.

Notes: Module **A1**. Reference: the MathWorks calibration page under **Resources**.

Next time

L03 · Rigid-body state, and two views

We stopped on the definition. Next week starts by using it: $SO(3)$ and $SE(3)$ are the groups we need, and the reason we wanted them is that you cannot correct a pose by adding to it.

- Where the arithmetic happens instead: the tangent space, $\exp$ and Log , and twists.
- Uncertainty as a tangent-space covariance, the lever arm, and the edge of a transform chain that drifts.
- Then two views: epipolar geometry, the essential and fundamental matrices, and the normalised eight-point algorithm.
- What makes a point matchable in the first place.

No class Monday Sep 7, Labor Day. We next meet Wednesday Sep 9.